

A Taste of Your Own Medicine: Design and Architect Infrastructure for an Organization

Project Report: Designing AWS Architecture for Organizations

Your Information

Your Name

Date of Submission

Course Title: "Designing AWS Architecture for Organizations"

Executive Summary

Excipient Technologies, a leading pharmaceutical maker, is consolidating its AWS network architecture.

- The company's network includes:
 - An AWS Control Tower-based network.
 - A standalone AWS account created by a project manager.
 - An AWS Organizations-based network from an acquired business.
- Your role as the senior cloud architect is to integrate these environments, establish network connectivity, enforce governance, and implement cost controls.
- Key Goals:
 - Integrate standalone and acquired accounts.
 - Enable secure and reliable interconnectivity.
 - Establish tagging strategies and governance.
 - Design cost management tools.

Multi-Account AWS Architecture Design (Step 1)

Objective: Design a multi-account architecture and integration strategy.

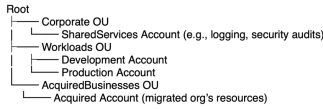
OU Hierarchy

- **Scenario Details:**
 - Design the organizational structure for Excipient Technologies. The company's hierarchy must include:
 - **Corporate OU:** Centralized services (e.g., logging and auditing).
 - **Workloads OU:** Environments for applications like Development and Production.
 - **Acquired Businesses OU:** Temporary holding area for acquired accounts.
- **Instructions:**
 - Create a diagram representing the proposed OU hierarchy.

- Include a brief explanation for each OU's role and purpose.

The following OU structure was designed to organize accounts for centralization and governance:

OU Diagram:



Explanation:

The diagram shows an AWS Organization with a Root node and two main Organizational Units (OUs): Corporate OU and Workloads OU.

- Root is the top-level container for all accounts and OUs, where global governance policies can be applied.
- Corporate OU contains accounts for shared services like security, networking, and centralized logging. It enforces strict policies to protect critical infrastructure.
- Workloads OU includes accounts for business applications and environments (e.g., dev, prod). Policies here balance security with flexibility for developers.

This multi-account setup improves security through isolation, enables targeted governance with Service Control Policies (SCPs), and uses tagging to track resources and costs efficiently.

Integration Strategy

Standalone Account Integration:

Provide the steps or objectives:

To integrate the standalone AWS account the following steps will be followed:

- Invite the standalone account into the AWS Organization using the AWS Organizations console.
- Enroll the account in AWS Control Tower using the Account Factory or the Account Enrollment workflow.
- Move the account into the appropriate Organizational Unit (e.g., Workloads OU or Corporate OU) based on its business function.
- Apply Service Control Policies (SCPs) and baseline guardrails to enforce compliance with governance standards.
- Enable monitoring and security services such as AWS Config, CloudTrail, and Security Hub for consistent auditing and visibility.

Acquired Organization Integration:

Provide the steps or objectives

The acquired business comes with its own AWS Organization. The following integration steps will be followed:

- Break apart the acquired AWS Organization, since Control Tower does not support nested orgs.
- Remove member accounts from the acquired org and invite them individually into the primary Organization under Control Tower.
- Place these accounts temporarily into the Acquired Businesses OU for staged onboarding and evaluation.
- Enable Control Tower landing zone setup on each newly joined account.
- Conduct resource and policy audits, and eventually reorganize or merge accounts into the permanent OUs (e.g., Production, Development).

- Enforce the standard SCPs, tagging policies, and monitoring configurations.

The following governance mechanisms will be enforced:

- Service Control Policies (SCPs):
 1. Deny creation of resources without mandatory tags.
 2. Deny use of unapproved regions.
 3. Restrict IAM role creation to Admins only.
- AWS Config Rules:
 1. Detect and report untagged or misconfigured resources.
 2. Ensure logging and encryption are enabled across all accounts.
- Centralized Logging:
 1. Send CloudTrail logs and Config data to a centralized S3 bucket in the Corporate OU.
- Security Hub + GuardDuty:
 1. Enable across all accounts for unified threat detection and compliance insights.

Network Connectivity Plan (Step 2)

Objective: Establish VPC peering connectivity between accounts.

1. Network Architecture

- **Scenario Details:**
 - The company has applications distributed across VPCs in different accounts. Secure connectivity must be established for inter-application communication.
 - Example Resources:
 - **Account A:** Hosts the company's database in a private VPC (CIDR: 10.0.0.0/16).
 - **Account B:** Hosts an analytics application in a separate VPC (CIDR: 192.168.0.0/16).

2. VPC Peering Setup

- **Instructions:**
 - Create VPC peering connections between Account A and Account B.
 - Update route tables in both VPCs to allow traffic flow.
 - Configure security groups to permit communication between instances.

3. Verification

- **Instructions:**
 - Launch EC2 instances in each VPC.
 - Test connectivity using tools like `ping` or `curl`.
 - Enable VPC Flow Logs to monitor traffic.

Deliverables:

- Connectivity deliverables :
 - Details of VPC peering setup (connection IDs, routes).
 - Screenshots of successful connectivity testing.
 - Logs showing traffic flow [optional]

[Insert VPC Diagram/description]**Objective:**

Establish network connectivity between two EC2 instances located in separate AWS regions using VPC Peering. Validate connectivity via ping over private IP.

1. VPC Peering Configuration

- VPC 1:
 - Name: **db-vpc-east**
 - Region: US East (N. Virginia)
 - CIDR: **10.0.0.0/16**
- VPC 2:
 - Name: **analytics-vpc-west**
 - Region: US West (N. California)
 - CIDR: **192.168.0.0/16**
- Peering Connection:
 - ID: **pcx-05886d9f4469fd783**
 - Status: **Active**

Route Table Updates:

Both VPCs' route tables were updated to allow routing to each other:

VPC	Destination CIDR	Target
East	192.168.0.0/16	Peering connection
West	10.0.0.0/16	Peering connection

2. Security Group Configuration

- Inbound Rules (for both instances):
 - Type: ICMP (Ping)
 - Source: The private IP or CIDR of the other instance's VPC (e.g., **10.0.0.0/16** ↔ **192.168.0.0/16**)
 - Port: N/A (for ping)
- Outbound Rules:
 - All traffic allowed (default)

3. EC2 Instances Launched

- ping-test-east
 - Region: N. Virginia
 - Private IP: **10.0.1.84**
- ping-test-west
 - Region: N. California
 - Private IP: **192.168.1.137**

Both instances were launched in public subnets with Internet Gateways attached for management access. EC2 Instance Connect was used for terminal access.

Governance with Tagging Strategies (Step 3)

Objective: Define and enforce a tagging strategy using SCPs and AWS Config rules.

1. Tagging Strategy

- **Scenario Details:**

- Excipient Technologies uses tagging for:
 - Tracking environments (Development, Production).
 - Assigning cost centers (e.g., R&D, Marketing).
 - Identifying resource owners.

- **Instructions:**

- Create a table specifying:
 - Tags (e.g., `Environment`, `CostCenter`, `Owner`).
 - Example values for each tag.
 - Purpose of each tag.

2. Enforcing Tagging Compliance

- **Instructions:**

- Write an SCP to deny the creation of resources without required tags.
- Draft an AWS Config rule to detect non-compliant resources.

Deliverables:

- Tagging strategy table.
- SCP JSON example.
- AWS Config rule JSON example.

Tagging Strategy Table

Category	Key	Value	Purpose
Cost Allocation	CostCenter	CC-001	Track resource costs by internal department or project
Ownership	Owner	<i>mahah@company.com</i>	Identify the person or team responsible for the resource
Environment	Environment	AnalyticsApp	Group resources under a specific application or initiative
Compliance	Confidentiality	High	Mark sensitivity level for compliance and security reviews

Enforcing Tagging Compliance

SCP Policy (JSON) :

Enter your JSON SCP Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyCreateWithoutTags",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEqualsIfExists": {
          "aws:RequestTag/Environment": ["Development", "Production"],
          "aws:RequestTag/CostCenter": ["R&D", "Marketing"],
          "aws:RequestTag/Owner": "*"
        }
      }
    }
  ]
}
```

This SCP denies creation of any resource unless all required tags are present with valid values.

AWS Config Rule (JSON) :

Enter your AWS Config Rule

```
json
{
  "ConfigRuleName": "RequiredTags",
  "Source": {
    "Owner": "AWS",
    "SourceIdentifier": "REQUIRED_TAGS"
  },
  "InputParameters": {
    "tag1Key": "Environment",
    "tag2Key": "CostCenter",
    "tag3Key": "Owner"
  },
  "Scope": {
    "ComplianceResourceTypes": ["AWS::AllSupported"]
  }
}
```

This rule checks if the specified tags exist on all supported AWS resource types and marks non-compliant ones.

Cost Management Plan (Step 4)

5.1 Budget Configuration

Budgets:

Enter the Budgets

- Budget Name: MonthlyEC2Budget
- Service: EC2
- Amount: \$50 USD per month
- Filter: Tagged with Environment = Development
- Time Period: Monthly
- Thresholds: 80%, 90%, 100%

Actions:

Enter the actions

- Action Type: Notification
- Recipients: billing-team@example.com
- At 80% Usage: Warning email to Owner
- At 90% Usage: Notify CostCenter heads
- At 100% Usage: Trigger alert to Finance admin